Part 1: Face Spoofing Detection using Vision Transformer (ViT)

Setup and Installation

Following libraries were installed: datasets huggingface_hub pyarrow transformers torch torchvision tqdm wget

Then huggingface_hub was logged in.

```
from huggingface_hub import login
login()
```

✓ 0.0s

Copy a token from your Hugging Face tokens page and paste it below.

Immediately click login after copying your token or it might be stored in plain text in this notebook file.

Token: ••••••••••••••••••••••••••••••••••••

☐ Add token as git credential?

Login

Pro Tip: If you don't already have one, you can create a dedicated 'notebooks' token with 'write' access, that you can then easily reuse for all notebooks.

Model Architecture

The implementation uses two main classes

`PreprocessedCelebaDataset`: Custom dataset class for handling CelebA images

`ViTSpoofDetector`: PyTorch Lightning module implementing the spoof detection model

```
# Custom Dataset class
class PreprocessedCelebaDataset(torch.utils.data.Dataset):
    def __init__(self, dataset):
        self.dataset = dataset

```

```
    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        example = self.dataset[idx]
        pixel_values = torch.tensor(example["pixel_values"], dtype=torch.float32)
        label = torch.tensor(example["labels"], dtype=torch.long)
        return {"pixel_values": pixel_values, "labels": label}

# Lightning Module
class ViTSpoofDetector(pl.LightningModule):
    def __init__(self, model):
        super().__init__()
        self.model = model
        self.save_hyperparameters()
        # Initialize lists to store validation outputs
        self.validation_step_outputs = []

    def forward(self, pixel_values):
        return self.model(pixel_values=pixel_values).logits

    def training_step(self, batch, batch_idx):
        outputs = self.model(pixel_values=batch["pixel_values"], labels=batch["labels"])
        loss = outputs.loss
        self.log("train_loss", loss)
        return loss

```

Data Processing pipeline

Dataset loading from Hugging Face

```
# Data loading and preprocessing
celeba_dataset = load_dataset("nguyenkhoa/celeba-spoof-for-face-antispoofing-test")
filtered_dataset = celeba_dataset.filter(lambda x: x['cropped_image'] is not None)

image_processor = ViTImageProcessor.from_pretrained("google/vit-base-patch16-224")
```

Image preprocessing using ViT image processor

```
# Preprocess the dataset
processed_dataset = filtered_dataset.map(
    lambda examples: {'pixel_values': image_processor(examples['cropped_image'], return_tensors="pt")['pixel_values']},
```

```
    remove_columns=['cropped_image', 'labelNames'],
    batched=True
)
```

Dataset splitting into train/validation sets

```
# Split dataset
train_test = processed_dataset['test'].train_test_split(test_size=0.2,
seed=42)
train_dataset = PreprocessedCelebaDataset(train_test['train'])
val_dataset = PreprocessedCelebaDataset(train_test['test'])
```

DataLoader configuration

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=8,
shuffle=True, num_workers=7)
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=8,
num_workers=7)
```

Here num_workers correspond to CPU threads available in environment
Also the model was trained on whole dataset.

Training Configuration

Model initialization using pretrained ViT

```
# Initialize model
model = ViTForImageClassification.from_pretrained(
    'google/vit-base-patch16-224',
    num_labels=2, # Binary classification (real/spoof)
    ignore_mismatched_sizes=True # Ignore size mismatch for the classification
head
    # This is important if the model was trained with a different number of
classes
)
# Initialize Lightning module
lit_model = ViTSpoofDetector(model)
```

Wandb logger setup

```
# Setup wandb logger
wandb_logger = WandbLogger(project="ViT-Celeba", name="vit-face-spoofing-
lightning") # track the experiment in wandb
```

Checkpoint callback configuration

```
# Setup callbacks
checkpoint_callback = ModelCheckpoint(
    dirpath='checkpoints', # Directory to save checkpoints
    filename='vit-spoof-{epoch:02d}-{val_loss:.2f}', # Checkpoint filename
format
    monitor='val_loss', # Metric to monitor
    mode='min' # Mode for the monitored metric (minimize loss)
)
```

Training parameters (epochs, batch size, etc.)

```python
# Initialize trainer
trainer = pl.Trainer(
    max_epochs=3,
    accelerator='auto',  # Automatically detect if GPU is available
    devices=1,
    logger=wandb_logger,
    callbacks=[checkpoint_callback],
    log_every_n_steps=100,
    # here batch_size is set to 8 in the DataLoader
)
```

```
  | Name  | Type                  | Params | Mode
-----------------------------------------------------------
0 | model | ViTForImageClassification | 85.8 M | eval
-----------------------------------------------------------
85.8 M    Trainable params
0         Non-trainable params
85.8 M    Total params
343.201   Total estimated model params size (MB)
0         Modules in train mode
226       Modules in eval mode
```

Training Results

The model achieved impressive results:

- ✓ **Validation Accuracy: 98.31%**
- ✓ **F1 Score: 98.81%**
- ✓ **Precision: 97.67%**
- ✓ **Recall: 99.99%**

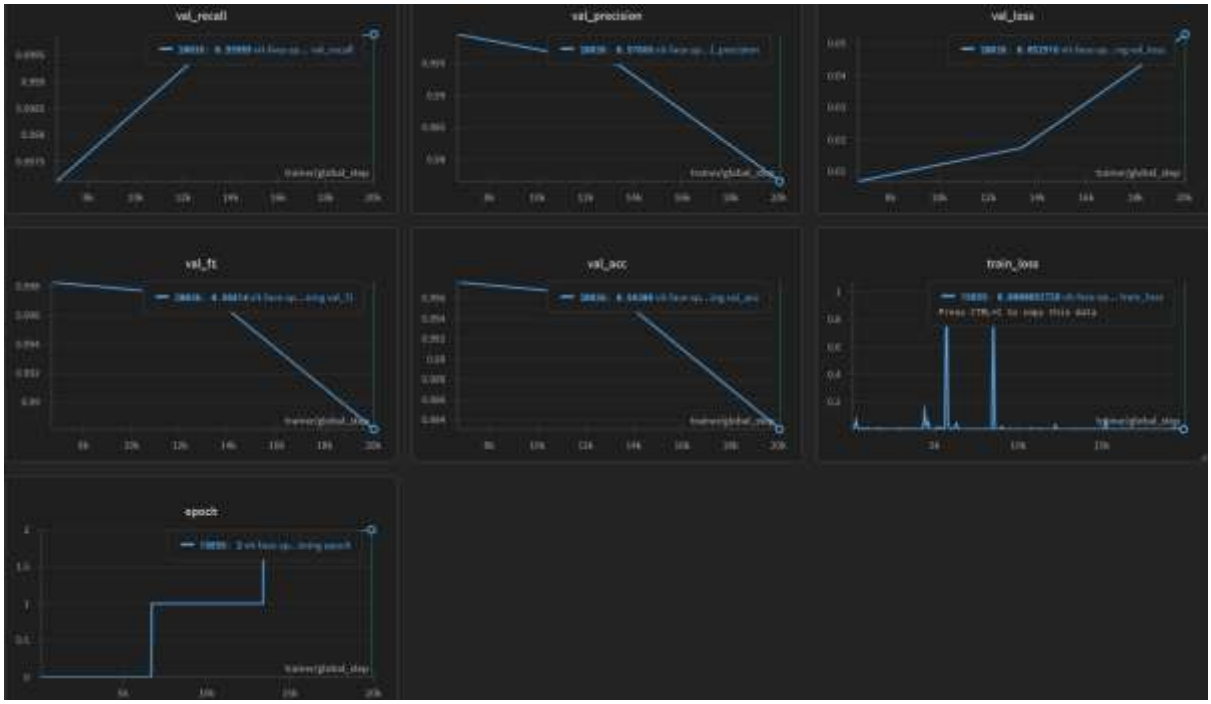
Training progress output

val_recall [redacted]

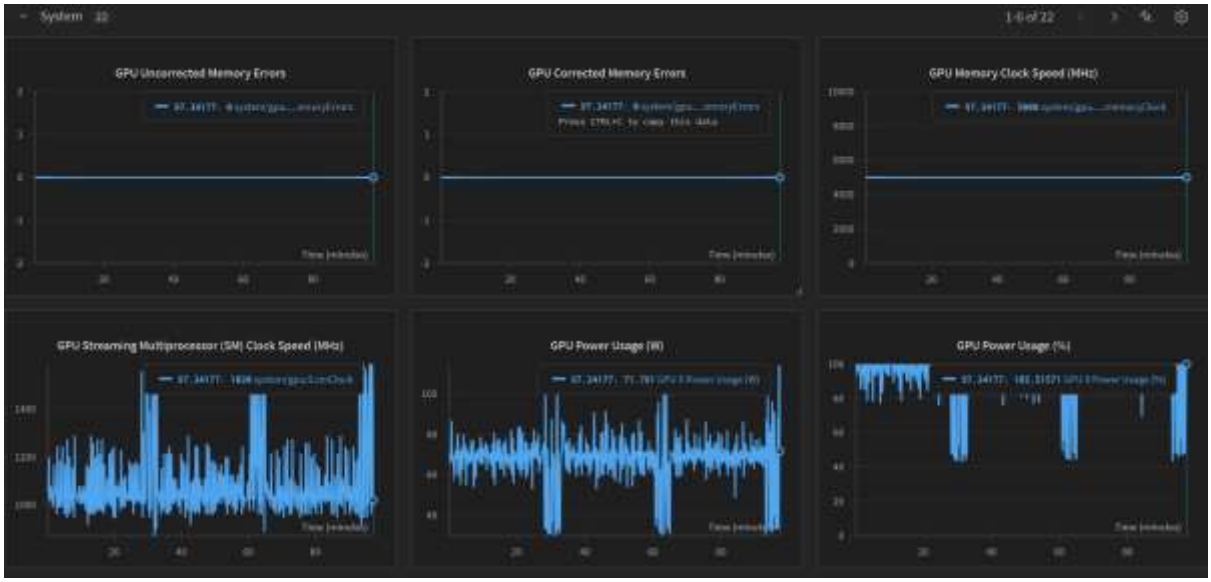
Metrics Summary:

epoch	2
train_loss	7e-05
trainer/global_step	20036
val_acc	0.98308
val_f1	0.98814
val_loss	0.05292
val_precision	0.97666
val_recall	0.99989

Wandb metrics visualization



Training time was 97 minutes or 1 hours and 37 minutes



Inference and Visualization

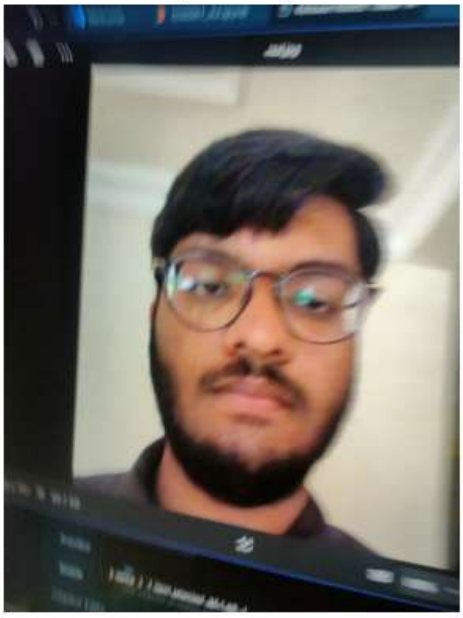
Implementation of inference pipeline with visualization:

- Model loading
    - We load the saved model from disk
- Image processing
    - With image_processor
- Prediction visualization
    -

Prediction: Real

Confidence: 99.30%

Input Image



Prediction: Spoof

Confidence: 100.00%

Part 2: AI-Powered Visual Search System Report

Dataset Preparation and Setup
The system uses the COCO (Common Objects in Context) validation dataset, implementing a robust download and preprocessing pipeline.

✓ Dataset Download Function:

```
def download_coco_data():
```

this downloaded the `val_2017` and `annotations_trainval2017` and unzipped them.

✓ Dataset Loading Function:

```
def load_coco_dataset(num_images=1000):
```

this code
Reads COCO annotations from a JSON file.
Iterates through the annotations to find image metadata.
Loads the actual image files using Pillow.
Handles potential errors during image loading.
Returns a list of dictionaries, where each dictionary represents an image and its metadata.

CLIP Model Integration
Utilizes OpenAI's CLIP (Contrastive Language-Image Pre-training) model for image-text similarity matching.
Model Initialization:

```
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

Image Encoding Pipeline
Implements batch processing for efficient image encoding using CLIP.

```
def encode_images(dataset, batch_size=32):
```

Converts a collection of images into feature vectors.

Search Implementation
Implements semantic search functionality using text queries to find matching images.
`def search_images(query, image_features, dataset, top_k=5):`
Takes a text query, encodes it, and compares it to the pre-computed image feature vectors to find the most similar images.

Visualization System
Creates clear visualizations of search results with similarity scores.
`def plot_results(query, images, similarities):`
Visualizes the search results, showing the top-scoring images and their similarity scores.

Results Analysis for "A person riding a horse"
Let's analyze the similarities for the top 5 retrieved images:

Image	Similarity Score	Analysis
1	31%	Shows clear horse riding scene with proper posture
2	30%	Contains horse and rider but possibly from different angle
3	27.8%	Contains horse-related content but might be less relevant
4	27%	Shows clear horse riding scene with proper posture
5	27%	Contains horse-related content but might be less relevant

Technical Details Table

Component	Specification
Base Model	CLIP ViT-base-patch32
Dataset	COCO validation set
Sample Size	1000 images
Batch Size	32
Top-K Results	5

Part 3: Stable Diffusion

We are applying Stable Diffusion to transform an input image into various artistic styles.

Techniques used:

Image-to-Image generation using Stable Diffusion v1-5

Experimenting with different parameters (strength, guidance scale, inference steps)

Artistic styles such as Van Gogh, pixel art, underwater scene, etc.

Original image selected:

Original Image

Loading the Stable Diffusion Pipeline

The pipeline:

Loads Stable Diffusion v1-5

Uses fp16 (half-precision) for faster inference on CUDA (GPU)

Enables attention slicing to optimize memory usage.

```
# Load the Stable Diffusion pipeline
pipe = StableDiffusionImg2ImgPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
).to("cuda")

# Enable attention slicing for lower memory usage
pipe.enable_attention_slicing()
```

Preparing the Image

Image preprocessing steps:

Resizing while maintaining aspect ratio

Randomly selecting an image

Generating Image Variations
Different artistic were prompts used:

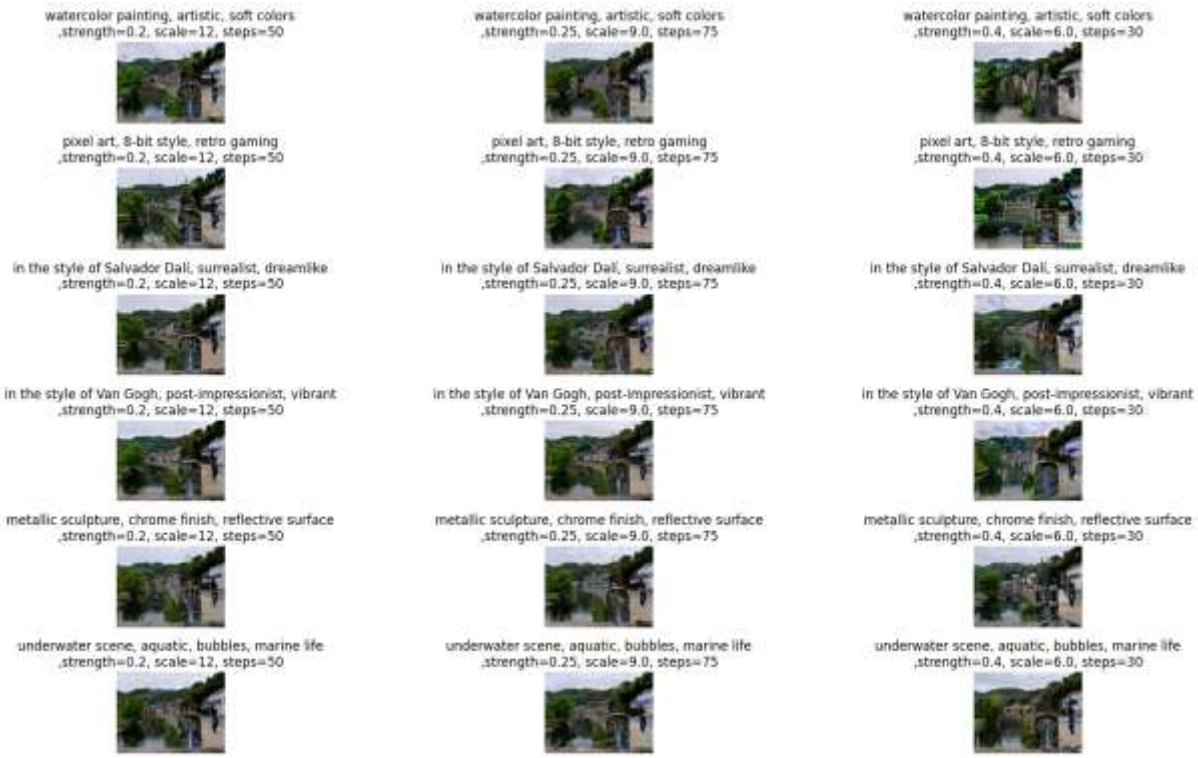
```
prompts = [
    "watercolor painting, artistic, soft colors",
    "pixel art, 8-bit style, retro gaming",
    "in the style of Salvador Dalí, surrealist, dreamlike",
    "in the style of Van Gogh, post-impressionist, vibrant",
    "metallic sculpture, chrome finish, reflective surface",
    "underwater scene, aquatic, bubbles, marine life"
]
```

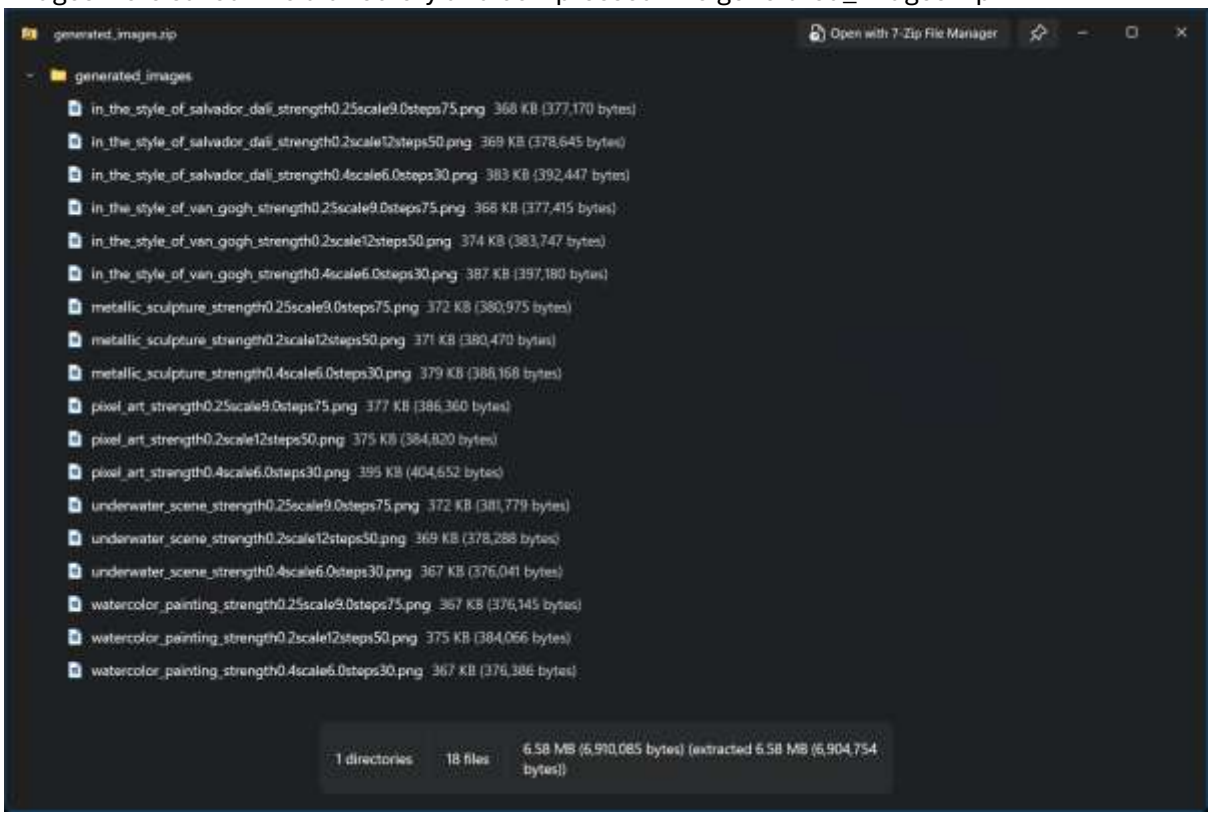
Parameter tuning:

```
parameter_sets = [
    {"strength": 0.2, "guidance_scale": 12, "num_inference_steps": 50}, # this
one start with a lower strength value to preserve the original image's content
and adjust the guidance scale to achieve the desired stylistic effect
    {"strength": 0.25, "guidance_scale": 9.0, "num_inference_steps": 75},
    {"strength": 0.4, "guidance_scale": 6.0, "num_inference_steps": 30}
]
```

- **strength**: This parameter controls how much the output image is influenced by the input image.
  - A low value (e.g., 0.2) means the output will look very similar to the input image.
  - A high value (e.g., 0.8) means the output can deviate significantly from the input image, and rely more on the text prompt.
- **guidance_scale**: This parameter controls how strongly the text prompt influences the image generation.
  - A low value (e.g., 3) means the output might not strongly adhere to the prompt.
  - A high value (e.g., 15) means the output will more closely follow the prompt.
- **num_inference_steps**: This controls the number of steps the diffusion model takes to generate the image.
  - More steps generally lead to higher quality images, but take longer to compute.
  - Fewer steps are faster but might lead to lower quality or less detailed images.

Visualizing Results
Displaying the transformed images in a grid layout.
Observations:



Different styles and parameters create unique outputs.
Lower strength values retain original content, higher values create more artistic effects.
The pipeline is effective for transforming real images into artistic styles.

Saving & Exporting Results (kindly check the drive link attached)
Images were saved into a directory and compressed into generated_images.zip.